Setting up Maven Java Project

- Configuring a Java project with Apache Maven involves **setting up the environment**, **creating the project structure**,
- defining the project's **pom.xml file**, and then using Maven **build commands**.

Step 1: Install Prerequisites and Maven

Before creating a project, ensure you have the necessary tools installed and configured.

- ✓ **Install the Java Development Kit (JDK):** Maven requires the JDK. You can download the latest version from Oracle's Java SE Downloads.
- ✓ **Set JAVA_HOME:** Configure the JAVA_HOME environment variable to point to your JDK installation directory.
- ✓ **Install Apache Maven:** Download the binary distribution from the official Apache Maven website.
- ✓ **Set Maven Environment Variables:** Extract the archive and set the MAVEN_HOME (or M2_HOME) environment variable to the installation path. Then, add the %MAVEN_HOME%\bin (Windows) or $MAVEN_HOME/bin (Unix/Mac) directory to your system's PATH variable.
- ✓ **Verify Installation:** Open a new terminal or command prompt and run mvn -version. You should see the installed Maven version information.

Step 2: Create a Maven Project

You can create a new project using the Maven Archetype plugin via the command line, which generates a standard project structure.

- ✓ Open your terminal and navigate to the directory where you want to create your project.
- ✓ Run the following command, replacing the groupId and artifactId with your desired project identifiers:
    - mvn archetype:generate -DgroupId=com.mycompany.app -DartifactId=my-app -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
- ✓ This command creates a new directory named my-app with the standard Maven directory layout:
    - ❖ src/main/java: Contains your application's source code.
    - ❖ src/test/java: Contains your test source code.
    - ❖ src/main/resources: For configuration files and other resources.
    - ❖ pom.xml: The Project Object Model file, central to the project configuration.

Step 3: Configure the pom.xml File

- The pom.xml is the core configuration file.
- The archetype command generates a basic one, which you can then customize.
- The minimal requirements for a pom.xml are modelVersion, groupId, artifactId, and version.

  A basic pom.xml looks like this:

```
<project xmlns="http://maven.apache.org"
      xmlns:xsi="http://www.w3.org"  xsi:schemaLocation="http://maven.apache.org ">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.mycompany.app</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging> <!-- Specifies the project will generate a JAR file -->
  <name>My App</name>
  <url>http://maven.apache.org</url>

  <dependencies>
    <!-- Example dependency for JUnit tests -->
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.11</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

- To find dependencies, you can search the public Maven Central Repository website (**mvnrepository.com**) and copy the XML snippet directly.

Step 4: Build the Project

- Once the project structure and pom.xml are set up, you can use Maven goals to build your project.
- Navigate to your project's root directory in the terminal (where pom.xml is located) and run the following commands:
  - ✓ **mvn compile**: Compiles the source code located in src/main/java, placing the .class files in the target/classes directory.
  - ✓ **mvn test**: Runs the unit tests from src/test/java.
  - ✓ **mvn package**: Compiles the code, runs tests, and packages the compiled code and resources into a distributable format (e.g., a .jar file) in the target directory.
  - ✓ **mvn install**: package the artifact and install it into your local Maven repository (~/.m2/repository), making it available for other local projects to use as a dependency.
  - ✓ **mvn clean install**: Deletes the target folder first to ensure a clean build before running install.